

PyQCU CUDA-C++ MultiGrid 各层算子构造

从奇子格 Schur 算子到 33 点 Galerkin 粗算子

代码与物理结构整理

PyQCU · Lattice QCD

2026-08-30

核心结论：层级由 Python 构造，CUDA-C++ 负责应用

Level 0

主求解使用奇子格 Schur 算子

$$S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}.$$

粗层

每次转移构造局部正交基 P_l , 并按

$$A_{l+1} = P_l^\dagger A_l P_l$$

投影为 33 点 stencil。

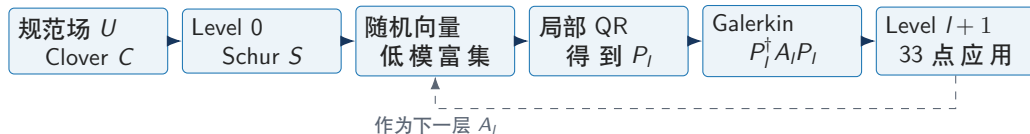
后端边界

C++ 初始化只解析层几何、分配求解向量；通过 `set_ptrs[30+4*1:34+4*1]` 接收 `lonv/hnn/hdg/sit`。

当前判断 (确证): 严格说, 当前 “CUDA-C++ 版 MultiGrid” 不是在 C++ 内部生成粗算子; 层级资产由 `build_schur_levels()` 生成或从 HDF5 缓存加载, `libqcu.so` 在 V-cycle 中消费这些资产。

来源: `pyqcu/cuda/_multi_gpu.py:63,111`; `cpp/cuda/qcu/src/apply_clover_multigrid.cu:11,55`; `cpp/cuda/qcu/include/lattice_clover_multigrid.h:2796`。

总流程：同一条 Galerkin 递归链贯穿全部层级



$$A_0 = S, \quad A_1 = P_0^\dagger S P_0, \quad A_2 = P_1^\dagger A_1 P_1, \quad \dots$$

每个转移只保存 $\{P_l, \text{on-site, nearest, diagonal}\}$ ，不保存完整稠密矩阵。

为什么递归可行？

P_l 具有 aggregate 局部支撑，且 A_l 只有有限邻接范围；因此投影后的 A_{l+1} 仍可用固定宽度 stencil 表示，随后重复同一构造流程。

来源：pyqcu/cuda/_multi_gpu.py:101-185; pyqcu/tools/_multigrid.py:738-770。图为实现逻辑抽象。

Level 0 不是粗化后的 Wilson 算子，而是奇子格 Schur 算子

$$D = \begin{pmatrix} D_{ee} & -\kappa H_{eo} \\ -\kappa H_{oe} & D_{oo} \end{pmatrix},$$
$$A_0 \equiv S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}.$$

物理图像

消去偶点后，奇点之间通过“奇 → 偶 → 奇”的两次 hopping 相连；偶点的局部 Clover 逆保留规范场背景和 spin-color 耦合。

对象	当前实现
输入/输出 自由度 几何 细层应用	奇子格向量 $12 = 4 \times 3$ $[X, Y, Z, T/2]$ C++
构造时 matvec	<code>fine_dslash_op</code> Python <code>op.matvec_parity</code> 或 <code>CudaSchurOp</code>

重要边界

源码中的 full-site 缓冲和最终 full-op 真残差检查不改变当前层级定义；主循环仍在 Level 0 调用 Schur 算子。

来源：lattice_clover_multigrid.h:815,4531,4824; _multi_gpu.py:497。

每条层间转移先构造局部低模基，再定义 $R_l = P_l^\dagger$

随机场 $v \in \mathbb{C}^{e_l \times X_l \times Y_l \times Z_l \times T_l}$

↓ 松弛 BiCGStab 近似求 $A_l^{-1}(A_l v)$

$v \leftarrow v - A_l^{-1}(A_l v)$

↓ 归一化并按 aggregate 重排

$[E_{l+1}, e_l, X_c, b_x, Y_c, b_y, Z_c, b_z, T_c, b_t]$

↓ 每个 aggregate 做 batched QR

P_l 的局部正交列向量

低模富集的含义

代码采用宽松相对容差，保留近零空间方向；这不是最终求解精度，而是为粗空间选择代表性基。

$$(R_l v)_J(c) = \sum_{x \in A(c), a} P_{J,a}^*(x) v_a(x),$$

$$(P_l c)_a(x) = \sum_J P_{J,a}(x) c_J(c).$$

局部正交性

QR 在每个 aggregate 内令

$$P_l^\dagger P_l = I$$

(各 aggregate 支撑互不重叠)，因此 restriction 不需要额外的全局 Gram 矩阵求逆。

来源: tools/_multigrid.py:355,69,112; src/multigrid.cu:13,73。

Galerkin 构造是逐个粗基向量探测，而不是显式形成完整矩阵

for each coarse site c and coarse basis J :

$$q \leftarrow e_J(c)$$

$$f \leftarrow P_I q$$

$$d \leftarrow A_I f$$

$$z \leftarrow P_I^\dagger d$$

将 z 在 c 及其 32 个邻接位置的块写入 stencil

$$A_{I+1} = P_I^\dagger A_I P_I.$$

列与块的对应

一个 (c, J) 探针给出 $P_I^\dagger A_I P_I$ 的一列；每个输出位置的系数是 $E_{I+1} \times E_{I+1}$ 复矩阵块。

来源: tools/_multigrid.py:691,738; lattice_clover_multigrid.h:892。

代码对象	数学对象
lonv	P_I
restrict	$P_I^\dagger$
matvec	A_I
sit/hnn/ hdg	A_{I+1} 的非零块

代码注释常写 P^T ；复数实现实际使用共轭转置 $P^\dagger$ 。

Schur 的两次 hopping 使粗层采用 33 点宽 stencil

$$\begin{aligned} & 1 \text{ onsite,} \\ & 4 \times 2 = 8 \text{ 同方向正/负邻居,} \\ & \binom{4}{2} \times 2 \times 2 = 24 \text{ 两方向弯折邻居,} \\ & N_{\text{stencil}} = 1 + 8 + 24 = 33. \end{aligned}$$

物理图像

D_{oo} 给出本地点；两次 hopping 回到原点、沿同一方向前进，或沿两个不同方向转弯。

来源：src/multigrid.cu:858,924; tools/_multigrid.py:738。

张量	语义与形状
sit	onsite: $[E, E, X, Y, Z, T]$
hop_nn	正/负邻居: $[2, 4, E, E, X, Y, Z, T]$
hop_diag	方向对与符号: $[2, 2, 6, E, E, X, Y, Z, T]$
应用式	$y_j(c) = \text{sit}_{je}(c)x_e(c)$ $+ \text{hop}_{je}(c)x_e(c \pm \hat{\mu})$ $+ \text{diag}_{je}(c)x_e(c \pm \hat{\mu} \pm \hat{\nu})$

33 是耦合位置数，不是自由度数；每个位置承载一个 $E \times E$ 块。

C++ 后端的职责：接收四类层级资产并执行宽 stencil

阶段	C++ 行为
init	读取每层 E, X, Y, Z, T , 分配求解向量
setops	保存四个外部指针
粗层算子	读取当前层 33 点 stencil
MPI 多 rank	先交换粗层 halo, 再启动 CUDA kernel
粗层求解	BiStabCG / fused coarse solve, 算子不变

来源：见末页源码索引：bridge:11; header:892。

指针协议

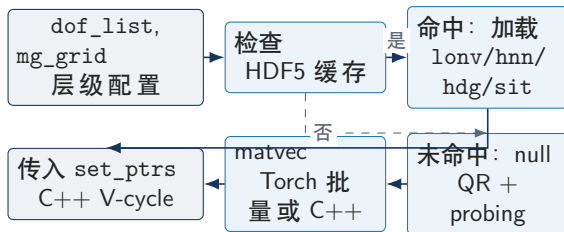
对每个父层 l , 令 $\text{base} = 30 + 4l$:

```
set_ptrs[base+0] → null_vecs,  
set_ptrs[base+1] → hop_nn,  
set_ptrs[base+2] → hop_diag,  
set_ptrs[base+3] → sit_packed.
```

实现分工（确证）

Python 侧决定 P_l 和 A_{l+1} 的数值内容；C++ 侧负责 GPU kernel、scratch、halo 和递归求解。

当前运行路径：缓存命中与构造加速只改变来源，不改变数学算子



默认路径的细节

batch_build=True 且传入 Python op 时，构造阶段主要使用 Torch 批量 Schur/stencil matvec；关闭批量模式或显式选择 C++ matvec 时，Level 0 使用 CudaSchurOp，后续层可使用 CudaCoarseSchurOp。两者都实现同一个 A_l 。

来源：_multi_gpu.py:501,539；_schur_op.py:68,149；run_qcu_mg.py:164。

V-cycle 中算子的使用顺序与最终边界

r_l : 父层残差

$$\downarrow \quad R_l = P_l^\dagger$$

r_{l+1} : 粗层右端

$$\downarrow \quad A_{l+1} x_{l+1} = r_{l+1}$$

x_{l+1} : 递归粗层解

$$\downarrow \quad P_l$$

e_l : 父层修正

$$\downarrow \quad x_l \leftarrow x_l + e_l, \text{ 重算 } r_l$$

层级关系

$$A_0 = S,$$

$$A_1 = P_0^\dagger S P_0,$$

$$A_2 = P_1^\dagger A_1 P_1.$$

对任意更深层重复同样模式。

边界

full-site 真残差与偶分量恢复只用于结果检查/输出，不参与粗算子构造；具体测例数值本次未重测。

来源: lattice_clover_multigrid.h:3348,4789,4846。

源码证据索引与范围说明

证据对象	层次	关键定位
Level 0 Schur	CUDA-C++ 核心	lattice_clover_multigrid.h:815
层级参数	CUDA-C++ 初始化	lattice_clover_multigrid.h: 2643
null vector	Python 工具	_multigrid.py:355
局部 QR/transfer	Python/CUDA	_multigrid.py:69; multigrid.cu:73
Galerkin probing	Python 工具	_multigrid.py:691
33 点 stencil	CUDA kernel	multigrid.cu:858
指针接线	C API bridge	apply_clover_multigrid.cu:11
缓存/构造	Python 驱动	_multi_gpu.py:490

来源：范围：相关入口、核心头文件、CUDA kernel、Python 构造器和缓存入口；源码关系与公式确证，性能与具体 lattice 数值质量未验证。